The Ch(e)at Code

Agent Patterns for Copilot Chat

ISSUE #013 • JUNE 23, 2026 • 🔧 PRACTICAL BUILD

Last issue, we named the Persistent Memory pattern. We laid out the three memory tiers, explained why raw transcripts fail, and defined the memory types that matter: facts, preferences, and episodes. This issue, we build it.

This issue's build: A complete Persistent Memory system in Copilot Studio — backed by Dataverse for storage, Power Automate for the remember/recall plumbing, and Azure OpenAI for intelligent fact extraction. Five components, one afternoon. The result: an agent that greets users with context, learns preferences over time, and never asks the same question twice.

🎮 AGENT SPOTLIGHT

The Memory-Enabled Agent

Copilot Studio + Dataverse + Power Automate + Azure OpenAI

The scenario: A customer's IT help desk agent handles hundreds of conversations per week. Every session starts cold — "What's your department? What project are you on? What did we discuss last time?" Users are frustrated. They told the agent their context yesterday. The agent forgot overnight. It feels like calling a support line and getting a different person every time — one who never reads the notes.

The solution: A Copilot Studio agent with a Dataverse-backed memory layer. When a user states a preference or shares a fact, the agent calls a "Remember" plugin to save it. On every new conversation, a "Recall" topic loads the user's stored memories into the system prompt. The agent greets with context: "Welcome back! Last time we were working on Project Falcon — want to pick up where we left off?"

Why this stack: Dataverse gives you row-level security, tenant data policies, and zero infrastructure management. Power Automate provides

the plugin actions without custom code. Azure OpenAI handles intelligent fact extraction so the agent knows *what* to remember without explicit user instructions.

🧩 **PATTERN BREAKDOWN**

Building Persistent Memory

Copilot Studio + Dataverse + Power Automate + Azure OpenAI

Why this matters: Issue #012 explained the pattern conceptually. Now we wire it up. The build has five components, each self-contained and independently testable. You can ship Component 1–3 in an afternoon and add 4–5 in a follow-up sprint.

END-TO-END MEMORY FLOW

```
User says something worth remembering
  ↓
Agent detects preference/fact via system prompt instruction
  ↓
Remember plugin (Power Automate) → upserts to Dataverse
  ↓
AgentMemory table stores key-value with type & confidence
  ↓
Next session: Recall topic queries Dataverse for user's memories
  ↓
Top memories injected into system prompt as context
  ↓
Agent greets with personalized context & tailored responses
```

Five components, in build order:

1. **Dataverse Memory Table.** Create a custom table called `AgentMemory` with these columns: `user_id`, `memory_type` (fact / preference / episode), `key`, `value`, `confidence`, `created_at`, `updated_at`, `expires_at`. Add a

composite index on `user_id` + `memory_type` for fast retrieval. Enable row-level security so users can only see their own memories. This takes about 5 minutes in the Dataverse maker portal.

2. **"Remember This" Plugin Action.** Build a Power Automate cloud flow exposed as a Copilot Studio plugin action. Input: an extracted fact as a key-value pair (e.g., `key: "preferred_format"`, `value: "bullet points"`). The flow performs an upsert: if a row with that key already exists for the user, it updates the value and timestamp; if not, it inserts a new row. Add this system prompt instruction to your agent: *"When the user states a preference or shares a fact about their work, call the Remember action to save it for future conversations."* Build time: ~20 minutes.

3. **"Recall" Greeting Topic.** Create a Copilot Studio topic that fires on every conversation start. It triggers a Power Automate flow that queries Dataverse for the user's stored memories — filtered to the top 10 most recent, highest-confidence facts. The flow returns a JSON array that gets injected into the system prompt as context: *"Known facts about this user: [list]."* The agent uses this to greet with context and avoid re-asking known information. Build time: ~15 minutes.

4. **Memory Hygiene.** Build a scheduled Power Automate flow that runs weekly: (a) delete all memories past their `expires_at` date, (b) summarize episodes older than 30 days into single-sentence summaries to save token budget, (c) flag memories with low confidence for review. Also add a user-facing agent action: "Forget everything about me" → deletes all rows matching the user's `user_id`. This is your GDPR compliance layer.

5. **Memory-Aware Responses.** The final piece is prompt engineering. Add this instruction to your agent's system prompt: *"Use the known facts to tailor your response. Do not ask the user to repeat information you already know. If you know their preferred format, use it. If you know their project, reference it."* The agent now uses recalled memories to personalize format preferences, project context, and role-based content — without the user lifting a finger.

Key insight: The intelligence is in the *extraction*, not the storage. The Remember plugin works best when paired with Azure OpenAI to intelligently parse what's worth saving from a conversation turn. Without AI extraction, you're relying on the user to explicitly say "remember this" — and they won't. The system prompt instruction tells the agent to proactively detect facts and preferences, then call the Remember action autonomously.

⚠ **Watch for memory conflicts:** If a user says "I prefer bullet points" on Monday and "use numbered lists" on Wednesday, the upsert handles it — the latest value wins. But if the agent extracts *two different keys* for the same

concept (`preferred_format` vs. `output_style`), you get conflicting memories. Normalize your key names in the extraction prompt to prevent this. Provide example keys in the system prompt.

🛠️ **TRY THIS NOW**

Give Your Agent a Memory in One Afternoon

1. **Create the Dataverse table.** Open the Power Apps maker portal, create a new table called `AgentMemory`, and add the columns: `user_id`, `memory_type`, `key`, `value`, `confidence`, `created_at`, `updated_at`, `expires_at`. Enable row-level security. Five minutes with the template.

2. **Build the "Remember" plugin action.** Create a Power Automate cloud flow: input is a key-value pair from the agent, action is an upsert to the `AgentMemory` table. Expose it as a plugin in Copilot Studio. Add the system prompt instruction telling the agent when to call it. Twenty minutes.

3. **Build the "Recall" greeting topic.** Create a topic triggered on conversation start. It calls a Power Automate flow that queries Dataverse for the user's top 10 memories and returns them as JSON. Inject into the system prompt. Fifteen minutes.

4. **Test the memory loop.** Tell the agent your name and your project. Close the conversation. Reopen it. Does the agent greet you by name and reference your project? If yes — you've built persistent memory. If not, check that the Recall topic is firing on conversation start and that the Remember plugin is writing to Dataverse.

💡 **The line that opens doors:** "What if your agent remembered every user's preferences, project context, and past decisions — so no one ever had to repeat themselves?"

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution